P.Vishwa Nandan(192473015)

TASK 5 – Apply Static Members and the Object Life Cycle

Marks: 3

This task explains how static variables, methods and blocks are used in the Campus Smart-Parking System. It also covers how objects are created, used and removed from memory.

The assignment has four main discussion areas and an alternative concept with complexity analysis. CO1 JAVA

1. Static Variables

A static variable belongs to the class rather than an individual object.

In our parking system, a **ticket counter** is a good example of a static variable.

Whenever a new parking ticket is generated, the ticket counter increases by one.

Example:

```
class ParkingFacility {


   static int ticketCounter = 1000;


   public static int generateTicketNumber() {
      ticketCounter++;
      return ticketCounter;
   }


   public static void main(String[] args) {


      System.out.println(generateTicketNumber());
      System.out.println(generateTicketNumber());
      System.out.println(generateTicketNumber());
   }
}
```

Output:

1001

1002

1003

Here, ticketCounter is static, so all ParkingFacility objects share the same counter.

Why is static used?

If we use an instance variable for the ticket counter, every ParkingFacility object gets a separate counter. This may generate duplicate ticket numbers.

Therefore, a static counter is preferred when ticket numbers must be unique across the campus.

Other examples:

- totalOccupancy can be static if it tracks the combined occupancy of the entire campus.
- Individual facility occupancy should be an instance variable because each facility has its own occupied slots.

2. Static Methods and Static Blocks

A. Static Methods

Static methods belong to the class and can be called without creating an object.

Example:

```
class ParkingFacility {

   static int ticketCounter = 1000;

   static int generateTicketNumber() {
      return ++ticketCounter;
   }

   public static void main(String[] args) {

      int ticket = ParkingFacility.generateTicketNumber();

      System.out.println("Ticket Number: " + ticket);
```

```
    }
}
```

Output:

Ticket Number: 1001

The generateTicketNumber() method is static because it operates on the shared ticket counter.

B. Static Block

A static block is used to initialize static variables. It executes only once when the class is initialized.

Example:

```
class ParkingFacility {

    static int ticketCounter;

    static {
        ticketCounter = 1000;
        System.out.println("Parking System Initialized");
    }

    public static void main(String[] args) {

        System.out.println("Ticket Counter: " + ticketCounter);
    }
}
```

Output:

Parking System Initialized

Ticket Counter: 1000

The static block initializes the ticket counter before the main method executes.

3. Object Life Cycle and Garbage Collection

An object is created using the new keyword and stored in heap memory.

In our parking system, a ParkingTicket object is created when a vehicle enters the parking facility.

Example:

ParkingTicket ticket = new ParkingTicket();

The object is used to store ticket information such as ticket number, vehicle details and entry time.

Object life cycle:

1. **Object Creation:** A ParkingTicket object is created when a vehicle enters.
2. **Object Usage:** The ticket stores vehicle information while it is parked.
3. **Vehicle Exit:** When the vehicle leaves, the parking slot is released and the active ticket is removed from the system.
4. **Garbage Collection:** When the ticket object is no longer reachable through any reference, it becomes eligible for garbage collection.

Java automatically manages memory using the Garbage Collector.

An object does not get deleted immediately when a vehicle leaves. It becomes eligible for garbage collection only when no reachable references to it remain.

If the system stores the ticket in parking history, that reference may keep the object reachable.

4. Passing and Returning Objects from Methods

In Java, objects can be passed to methods using reference variables and returned from methods.

Example:

```
class Vehicle {


   String vehicleNo;


   Vehicle(String vehicleNo) {
      this.vehicleNo = vehicleNo;
   }
}


class ParkingTicket {
```

```
    Vehicle vehicle;

    ParkingTicket(Vehicle vehicle) {
        this.vehicle = vehicle;
    }
}

class Main {

    static ParkingTicket generateTicket(Vehicle v) {
        return new ParkingTicket(v);
    }

    public static void main(String[] args) {

        Vehicle v = new Vehicle("TN01AB1234");

        ParkingTicket t = generateTicket(v);

        System.out.println(t.vehicle.vehicleNo);
    }
}
```

Output:

TN01AB1234

Explanation:

1. A Vehicle object is created.
2. The vehicle reference is passed to the generateTicket() method.
3. The method creates a ParkingTicket object.

4. The ParkingTicket object is returned to the main method.

Java passes arguments by value. When an object is passed, a copy of its reference value is passed to the method.

---

5. Alternative Concept and Complexity Analysis

The assignment asks what happens if the static ticket counter is changed into an instance variable of ParkingFacility.

Consider a university with two parking facilities:

- North Gate Parking.
- South Gate Parking.

A. Static Counter

When the ticket counter is static, both facilities share one counter.

Example:

North Gate → Ticket 1001

South Gate → Ticket 1002

North Gate → Ticket 1003

South Gate → Ticket 1004

This prevents duplicate ticket numbers between facilities running in the same Java application.

B. Instance Counter

If the ticket counter is an instance variable, each facility maintains its own counter.

Example:

North Gate → Ticket 1001

South Gate → Ticket 1001

North Gate → Ticket 1002

South Gate → Ticket 1002

This may generate duplicate ticket numbers.

To avoid this problem, each facility can use a unique facility identifier along with its counter.

Example:

NORTH-1001

SOUTH-1001

Alternatively, a shared static counter can be maintained across both facilities.

Complexity Analysis

Let:

- k = Number of parking facility objects.
- n = Number of parking tickets.

1. Static Counter

- Only one counter is shared by all facility objects.
- Space complexity: O(1).
- Increment operation: O(1).

2. Instance Counter

- Each facility object maintains its own counter.
- Space complexity: O(k).
- Increment operation: O(1) per facility.

3. Parking Ticket Objects

The parking ticket objects require separate memory.

- Space complexity for n ticket objects: O(n).

The ticket objects occupy O(n) space regardless of whether the counter is static or an instance variable.

Comparison

A static counter requires memory for only one shared counter, whereas an instance counter requires a separate counter for every facility object.

Therefore, a static counter is more suitable for generating campus-wide unique ticket numbers.

6. Conclusion

Static variables are useful for sharing common information such as ticket counters across multiple parking facilities. Static methods perform class-level operations, while static blocks initialize shared data. ParkingTicket objects are created when vehicles enter and become eligible for garbage collection when they are no longer reachable. A static ticket counter is preferred because it avoids duplicate ticket numbers across different facilities